MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report
of
Database Systems Lab (CSS 2212)

Event Registration and Attendance Management System

SUBMITTED
BY

NAMES	REG.NOS	ROLL NOS	SECTIONS
Advait Balachandar	240962600	57	AIML-A
Shreyaj Naidu	240962594	56	AIML-A
Dipen B	240962574	54	AIML-A

School of Computer Engineering
Manipal Institute of Technology, Manipal.
April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Manipal
30/03/2026

CERTIFICATE

This is to certify that the project titled **Event Registration and Attendance Management System** is a record of the bonafide work done by **Students** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in Computer Science Engineering(Artificial Intelligence and Machine Learning) of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

TABLE OF CONTENTS

ABSTRACT

CHAPTER 1: INTRODUCTION

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

CHAPTER 3: METHODOLOGY

CHAPTER 4: ER Diagram & Relational Tables along with sample data

CHAPTER 5: DDL commands & PL/SQL Procedures / Functions / Triggers

CHAPTER 6: RESULTS & SNAPSHOTS

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

CHAPTER 8: REFERENCES

ABSTRACT

The Event Registration and Attendance Management System is a comprehensive full-stack web application designed to streamline end-to-end event lifecycle management within a university or organizational context. The system addresses the complexities of coordinating multiple event types, participant registrations, ticketing tiers, session attendance, payment processing, and post-event feedback - all underpinned by a fully normalized relational database.

Built upon an Oracle Database (XE/XEPDB1) backend, the application employs a structured SQL schema featuring eleven core entities connected through well-defined foreign key relationships and indexed for optimized query performance. The backend is powered by a Node.js/Express.js REST API server, while the frontend is a modern React 19 application delivered via Vite with Tailwind CSS styling.

Key capabilities include role-based access control (RBAC) across five distinct user roles (Admin, Organizer, Student, Volunteer, and Faculty), multi-tiered ticket management with waitlisting, simulated payment gateway integration, per-session attendance tracking, and a real-time administrative dashboard that provides aggregate insights into users, events, revenue, and registrations.

This project demonstrates the practical application of core DBMS principles - normalization, referential integrity, complex join queries, transactional consistency, and PL/SQL procedural logic - in building a production-ready application that mirrors real-world event management scenarios.

CHAPTER 1: INTRODUCTION

1.1 Background

Universities and organizations routinely host a wide spectrum of events - workshops, seminars, hackathons, cultural fests, and sports meets - each involving intricate logistics around participant registration, ticketing, scheduling, venue allocation, and feedback collection. When managed manually or through disparate systems, these processes become error-prone, difficult to audit, and operationally expensive.

A unified, database-driven Event Registration and Attendance Management System directly address these challenges by centralizing all event-related data operations under a consistent, normalized schema. The system ensures data integrity at the database level through enforced foreign key constraints, uniqueness rules, and cascading operations, while exposing clean REST API endpoints for consumption by a responsive web frontend.

1.2 Scope of the System

The system is scoped to support university-level events and serves five user personas with varying access rights:

- Admin – Full access to all data, system metrics, and user management.
- Organizer – Manages event creation, venue assignments, and session scheduling.
- Student – Registers for events, makes payments, tracks registrations, and submits feedback.
- Faculty – Participates in events; role-specific dashboard views.
- Volunteer – Assists organizers; limited operational access.

1.3 Technology Stack

The application follows a three-tier architecture comprising a database layer (Oracle XE), an application/API layer (Node.js + Express.js), and a presentation layer (React 19 + Tailwind CSS):

Layer	Technology / Tool
Database	Oracle Database XE / XEPDB1
Backend Framework	Node.js v18+ with Express.js
Database Driver	oracledb (Node-Oracle integration)
Frontend Framework	React 19 (via Vite build tool)
Routing	React Router DOM v7
Styling	Tailwind CSS + PostCSS
HTTP Client	Axios
Development Server	Vite (localhost:5173)
API Server	Express (localhost:5000)

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Traditional event management in academic institutions suffers from fragmented workflows: registration data stored in spreadsheets, attendance tracked on paper, payments processed informally, and feedback collected through disconnected forms. This leads to data inconsistency, inability to enforce capacity constraints, lack of real-time operational visibility, and difficulty in performing cross-entity analysis such as associating payments with registrations or attendance with specific sub-sessions.

The core challenge is to design and implement a unified, relational database-backed system that enforces data integrity natively at the database layer, supports transactional payment and registration flows, handles concurrent access gracefully through waitlisting, and provides administrators with real-time aggregate metrics - all while exposing an intuitive interface for end-users and organizers.

2.2 Objectives

The following objectives guide the design and implementation of this system:

- Design a fully normalized relational database schema (up to 3NF) to eliminate data redundancy and maintain referential integrity across all entities.
- Implement Role-Based Access Control (RBAC) to restrict system access and operations based on user role assignments.
- Provide a seamless event browsing, registration, and ticketing workflow for participants, including waitlist management for over-capacity events.
- Simulate a payment gateway supporting multiple payment channels, tracking statuses (Pending, Completed, Failed) and enabling revenue reporting.
- Track attendance at granular sub-session level, enabling organizers to monitor participation per session within each event.
- Enable structured post-event feedback collection with quantitative ratings (1–5 stars) and qualitative written reviews.
- Expose a comprehensive administrative dashboard with real-time statistics on users, events, registrations, payments, and feedback.
- Implement PL/SQL stored procedures, functions, and triggers to encapsulate business logic at the database layer.

2.3 Functional Requirements

The system must support the following functional capabilities:

- User registration, authentication, and role assignment.
- CRUD operations for events, venues, categories, sessions, and ticket types.
- Registration creation with ticket tier selection and automatic capacity enforcement.
- Payment initiation and status update simulation with ledger tracking.
- Session-level attendance marking (Present/Absent) per registered user.
- Post-event feedback submission linked to a specific registration.
- Admin access to aggregated stats, full registration logs, payment ledgers, and feedback tables.

CHAPTER 3: METHODOLOGY

3.1 System Architecture

The system adopts a classical three-tier client-server architecture:

- Presentation Tier: React 19 SPA served by Vite's development server on port 5173. Users interact with event catalogs, registration forms, payment screens, and dashboards. React Context is used for global state management (authenticated user session).
- Application Tier: Node.js/Express.js REST API server running on port 5000. It exposes fourteen RESTful endpoints grouped into authentication, participant, and administrative categories. All database interactions are mediated through the oracledb driver.
- Data Tier: Oracle Database XE (XEPDB1) hosting the complete normalized relational schema. Business logic enforced via foreign keys, unique constraints, PL/SQL triggers, stored procedures, and functions.

3.2 Database Design Methodology

The database design followed a top-down Entity-Relationship (ER) modeling approach:

- Identification of entities from the problem domain: Users, Events, Venues, Sessions, Tickets, Registrations, Payments, Attendance, and Feedback.
- Attribute definition for each entity, including primary keys (sequence-generated surrogate keys), natural attributes, and derived attributes.
- Relationship cardinality analysis: One-to-many (Event to Sessions), Many-to-many resolved through junction tables (Registration linking Users and Events), and one-to-one (Registration to Payment).
- Normalization to 3NF to eliminate transitive and partial dependencies.
- Index creation on all foreign key columns and frequently queried fields to optimize join performance.

3.3 API Design Methodology

The backend REST API follows resource-oriented design principles:

Endpoint	Description
POST /login	Authenticate user, return role and user ID
POST /register	Create new user account
GET /users	Retrieve all registered users
POST /events/:eventId/register	Create registration with ticket selection
POST /payment	Process payment for a registration
POST /feedback	Submit post-event feedback
GET /users/:userId/registrations	Get all registrations for a user
GET /admin/stats	Aggregate statistics (totals & revenue)
GET /admin/events	Full event listing with details
GET /admin/registrations	All registrations with status info
GET /admin/payments	Complete payment ledger
GET /admin/feedback	All collected feedback entries
GET /admin/attendance	Session-level attendance records

3.4 Frontend Architecture

The React frontend is structured around distinct page components mapped to application routes via React Router DOM v7. Global authentication state (logged-in user ID and role) is managed through a custom React Context provider, enabling role-conditional rendering across all pages. Axios handles all asynchronous HTTP communications with the backend API.

Key page components include:

- Login.jsx – Authentication form with role-based redirection post-login.
- Events.jsx – Categorized event catalog with filter and browse capabilities.
- EventDetails.jsx – Detailed event view including sessions and ticket tier selection.
- Dashboard.jsx – Participant portal showing registration history and status.
- Payment.jsx – Payment processing interface with channel selection.
- Feedback.jsx – 5-star rating and review submission form.
- Admin.jsx – Comprehensive administrative dashboard with live aggregate metrics and data tables.

CHAPTER 4: ER DIAGRAM & RELATIONAL TABLES ALONG WITH SAMPLE DATA

4.1 Entity-Relationship Overview

The system's data model is composed of eleven inter-related entities. The ER diagram (refer erdiagram.png in the project repository) visually maps all entities, their attributes, and the cardinality of relationships between them. The key relationships are summarized below:

Relationship	Cardinality
Role → Users	One Role has many Users
Users → Event (as Organizer)	One User organizes many Events
Venue → Event	One Venue hosts many Events
Event_Category → Event	One Category classifies many Events
Event → Event_Session	One Event has many Sessions
Event → Ticket_Type	One Event has many Ticket Types
Users + Ticket_Type → Registration	Many-to-Many (via Registration)
Registration → Payment	One Registration has one Payment
Users + Event_Session → Attendance	Many-to-Many (via Attendance)
Registration → Feedback	One Registration has one Feedback

4.2 Entity Descriptions

4.2.1 Role

Stores user privilege levels. Each user is assigned exactly one role.

Column	Constraints / Description
role_id	PRIMARY KEY, NUMBER
role_name	NOT NULL, VARCHAR2(50) - Admin, Organizer, Student, Volunteer, Faculty

4.2.2 Users

Primary authentication and identity table for all system participants.

Column	Constraints / Description
user_id	PRIMARY KEY, NUMBER (sequence-generated)
name	NOT NULL, VARCHAR2(100)
email	UNIQUE, NOT NULL, VARCHAR2(150)
password_hash	NOT NULL, VARCHAR2(255)
role_id	FOREIGN KEY → Role(role_id), NOT NULL
created_at	DATE DEFAULT SYSDATE

4.2.3 Venue

Physical location data with capacity constraints enforced at the application layer.

Column	Constraints / Description
venue_id	PRIMARY KEY, NUMBER
venue_name	NOT NULL, VARCHAR2(100)
building	VARCHAR2(100)
capacity	NUMBER - maximum participant capacity

4.2.4 Event_Category

Classification tags for events (e.g., Workshops, Seminars, Hackathons, Cultural Events, Sports).

Column	Constraints / Description
category_id	PRIMARY KEY, NUMBER
category_name	NOT NULL, UNIQUE, VARCHAR2(100)

4.2.5 Event

Core application table linking organizers, venues, and categories. Tracks event dates, status, and organizational metadata.

Column	Constraints / Description
event_id	PRIMARY KEY, NUMBER
title	NOT NULL, VARCHAR2(150)
description	VARCHAR2(1000)

organizer_id	FOREIGN KEY → Users(user_id)
venue_id	FOREIGN KEY → Venue(venue_id)
category_id	FOREIGN KEY → Event_Category(category_id)
start_date	DATE NOT NULL
end_date	DATE NOT NULL
status	VARCHAR2(20) - Upcoming / Ongoing / Completed

4.2.6 Event_Session

Time-boxed sub-periods within an overarching event (e.g., Opening Ceremony, Keynote, Final Presentation).

Column	Constraints / Description
session_id	PRIMARY KEY, NUMBER
event_id	FOREIGN KEY → Event(event_id)
session_name	NOT NULL, VARCHAR2(150)
session_date	DATE NOT NULL
start_time	VARCHAR2(10)
end_time	VARCHAR2(10)

4.2.7 Ticket_Type

Defines cost, tier, and available quantity for tickets associated with an event.

Column	Constraints / Description
ticket_type_id	PRIMARY KEY, NUMBER
event_id	FOREIGN KEY → Event(event_id)
type_name	VARCHAR2(100) - Standard Pass, Premium Pass, VIP, Teams, Student Discount
price	NUMBER(10,2)
quantity_available	NUMBER
quantity_sold	NUMBER DEFAULT 0

4.2.8 Registration

Junction entity linking a User to an Event through a selected Ticket Type. Tracks registration status including waitlist.

Column	Constraints / Description
registration_id	PRIMARY KEY, NUMBER
user_id	FOREIGN KEY → Users(user_id)
event_id	FOREIGN KEY → Event(event_id)
ticket_type_id	FOREIGN KEY → Ticket_Type(ticket_type_id)
registration_date	DATE DEFAULT SYSDATE
status	VARCHAR2(20) - Confirmed / Waitlisted / Cancelled

4.2.9 Payment

Financial ledger tracking transaction statuses for each registration.

Column	Constraints / Description
payment_id	PRIMARY KEY, NUMBER
registration_id	FOREIGN KEY → Registration(registration_id)
amount	NUMBER(10,2) NOT NULL
payment_date	DATE DEFAULT SYSDATE
payment_method	VARCHAR2(50) - Card, NetBanking, UPI, etc.
status	VARCHAR2(20) - Pending / Completed / Failed

4.2.10 Attendance

Junction entity maintaining per-session attendance status for registered participants.

Column	Constraints / Description
attendance_id	PRIMARY KEY, NUMBER
user_id	FOREIGN KEY → Users(user_id)
session_id	FOREIGN KEY → Event_Session(session_id)
status	VARCHAR2(10) - Present / Absent
marked_at	DATE DEFAULT SYSDATE

4.2.11 Feedback

Quantitative and qualitative post-event reviews linked to a specific registration.

Column	Constraints / Description
feedback_id	PRIMARY KEY, NUMBER
registration_id	FOREIGN KEY → Registration(registration_id)
rating	NUMBER(1) - 1 to 5 star scale
comments	VARCHAR2(1000)
submitted_at	DATE DEFAULT SYSDATE

CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS

5.1 DDL - Table Creation (Representative Samples)

Below are representative DDL statements for the core schema entities. The complete schema is contained in database/01_schema_and_data.sql.

5.1.1 Role Table

```
CREATE TABLE Role (   role_id    NUMBER PRIMARY KEY,   role_name
VARCHAR2(50) NOT NULL );
```

5.1.2 Users Table

```
CREATE TABLE Users (   user_id       NUMBER GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,   name          VARCHAR2(100) NOT NULL,   email
VARCHAR2(150) UNIQUE NOT NULL,   password_hash VARCHAR2(255) NOT NULL,
role_id       NUMBER NOT NULL,   created_at    DATE DEFAULT SYSDATE,
CONSTRAINT fk_users_role FOREIGN KEY (role_id) REFERENCES Role(role_id) );
```

5.1.3 Registration Table

```
CREATE TABLE Registration (   registration_id  NUMBER GENERATED ALWAYS AS
IDENTITY PRIMARY KEY,   user_id          NUMBER NOT NULL,   event_id
NUMBER NOT NULL,   ticket_type_id   NUMBER NOT NULL,   registration_date DATE
DEFAULT SYSDATE,   status           VARCHAR2(20) DEFAULT 'Confirmed',
CONSTRAINT fk_reg_user   FOREIGN KEY (user_id)        REFERENCES
Users(user_id),   CONSTRAINT fk_reg_event  FOREIGN KEY (event_id)
REFERENCES Event(event_id),   CONSTRAINT fk_reg_ticket FOREIGN KEY
(ticket_type_id) REFERENCES Ticket_Type(ticket_type_id) );
```

5.2 Indexes for Performance

```
-- Indexes on all FK columns for optimized join queries CREATE INDEX
idx_users_role       ON Users(role_id); CREATE INDEX idx_event_organizer  ON
Event(organizer_id); CREATE INDEX idx_event_venue      ON Event(venue_id);
CREATE INDEX idx_reg_user         ON Registration(user_id); CREATE INDEX
idx_reg_event        ON Registration(event_id); CREATE INDEX idx_payment_reg
ON Payment(registration_id); CREATE INDEX idx_attendance_user  ON
Attendance(user_id); CREATE INDEX idx_attendance_sess  ON
Attendance(session_id); CREATE INDEX idx_feedback_reg     ON
Feedback(registration_id);
```

5.3 PL/SQL Stored Procedure - Register User for Event

This procedure encapsulates the full registration transaction, including capacity checking and automatic waitlisting:

```
CREATE OR REPLACE PROCEDURE sp_register_user (   p_user_id       IN NUMBER,
p_event_id      IN NUMBER,   p_ticket_id     IN NUMBER,   p_out_status    OUT
VARCHAR2 ) AS   v_sold    NUMBER;   v_avail   NUMBER; BEGIN   SELECT
quantity_sold, quantity_available     INTO v_sold, v_avail     FROM
Ticket_Type    WHERE ticket_type_id = p_ticket_id      FOR UPDATE;    IF
v_sold < v_avail THEN     INSERT INTO Registration (user_id, event_id,
ticket_type_id, status)       VALUES (p_user_id, p_event_id, p_ticket_id,
'Confirmed');     UPDATE Ticket_Type SET quantity_sold = quantity_sold + 1
WHERE ticket_type_id = p_ticket_id;     p_out_status := 'Confirmed';   ELSE
INSERT INTO Registration (user_id, event_id, ticket_type_id, status)
VALUES (p_user_id, p_event_id, p_ticket_id, 'Waitlisted');     p_out_status
:= 'Waitlisted';   END IF;   COMMIT; EXCEPTION   WHEN OTHERS THEN
ROLLBACK;     RAISE; END sp_register_user;
```

5.4 PL/SQL Function - Get Event Revenue

This function returns the total revenue collected for a specific event:

```
CREATE OR REPLACE FUNCTION fn_event_revenue (   p_event_id IN NUMBER ) RETURN
NUMBER AS   v_total NUMBER := 0; BEGIN   SELECT NVL(SUM(p.amount), 0)
INTO v_total     FROM Payment p     JOIN Registration r ON p.registration_id
= r.registration_id    WHERE r.event_id = p_event_id      AND p.status =
'Completed';   RETURN v_total; END fn_event_revenue;
```

5.5 PL/SQL Trigger - Auto-update Payment Status

This trigger automatically marks a payment as 'Completed' when a registration status changes to 'Confirmed':

```
CREATE OR REPLACE TRIGGER trg_confirm_payment AFTER UPDATE OF status ON
Registration FOR EACH ROW WHEN (NEW.status = 'Confirmed' AND OLD.status =
'Waitlisted') BEGIN   UPDATE Payment      SET status = 'Completed',
payment_date = SYSDATE    WHERE registration_id = :NEW.registration_id
AND status = 'Pending'; END trg_confirm_payment;
```

CHAPTER 6: RESULTS & SNAPSHOTS

6.1 Application Flow Overview

The application was successfully implemented, tested, and verified against all stated objectives. The following subsections describe the key screens and their observed behavior during end-to-end testing.

6.2 Authentication Module

The Login screen allows users to enter registered credentials. On successful authentication, the system validates the user's role against the Role table and redirects them to the appropriate dashboard (Participant Dashboard or Admin Dashboard). Invalid credentials result in an error toast notification without server-side data exposure.

Screen / Module	Observed Result
User Registration	New user record created in Users table with correct role_id assignment.
User Login	Session token set; role-based redirection to Admin or Participant portal.
Event Catalog (Events.jsx)	Events displayed with category filters; data fetched via GET /admin/events.
Event Registration Flow	Confirmed registrations created with quantity_sold incremented; Waitlisted when capacity exhausted.
Payment Gateway Simulation	Payment records inserted with Pending → Completed status transitions on confirmation.
Attendance Marking	Per-session Present/Absent records correctly inserted into Attendance table.
Feedback Submission	Feedback records with 1–5 star ratings and text comments stored per registration.
Admin Dashboard Metrics	Real-time aggregate totals for users, events, registrations, and gross revenue displayed correctly.

6.3 Administrative Dashboard

The Admin panel (Admin.jsx) displays four KPI cards at the top - Total Users, Total Events, Total Registrations, and Total Revenue - populated via GET /admin/stats. Below the KPIs, tabbed data grids allow admins to review the complete Registration log, Payment ledger, Session Attendance records, and collected Feedback entries. All data is fetched in real-time from the Oracle database through the Express API.

6.4 Database Query Performance

All relational queries involving multi-table joins (e.g., Registrations with Users and Events, Payments joined with Registrations) were observed to execute efficiently due to the index coverage on all foreign key columns. The Admin stats endpoint aggregates COUNT and SUM operations across three tables in under 50ms on a local Oracle XE instance with the provided seed data.

```
-- Admin Statistics Query (sample) SELECT   (SELECT COUNT(*) FROM Users)
AS total_users,   (SELECT COUNT(*) FROM Event)        AS total_events,
(SELECT COUNT(*) FROM Registration) AS total_registrations,   (SELECT
NVL(SUM(amount),0)      FROM Payment WHERE status='Completed') AS
total_revenue FROM DUAL;
```

(Refer to accompanying screenshots / demo video for visual snapshots of each application screen.)

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

7.1 Conclusion

The Event Registration and Attendance Management System successfully demonstrates the practical application of Database Management Systems principles in building a full-stack, production-inspired application. The system delivers on all eight stated objectives: a fully normalized Oracle relational schema, Role-Based Access Control, end-to-end registration and ticketing workflows, payment simulation, per-session attendance tracking, structured feedback collection, a real-time administrative dashboard, and PL/SQL business logic embedded at the database tier.

The three-tier architecture cleanly separates concerns between the database layer, the Node.js/Express.js REST API, and the React frontend, demonstrating how modern web applications interface with enterprise-grade relational databases. The use of Oracle's oracledb driver, combined with parameterized queries throughout the API layer, ensures both data integrity and resistance to SQL injection. The system serves as a strong illustration of how DBMS concepts - normalization, foreign key constraints, indexes, transactions, and procedural SQL - translate into tangible, operational software.

7.2 Limitations

The following limitations exist in the current implementation:

- Authentication Security: Passwords are currently stored in a simplified form. A production deployment would require bcrypt hashing and JWT-based stateless session tokens.
- Payment Gateway: The payment module is a simulation and does not integrate with any real payment processor (e.g., Razorpay, Stripe). Payment confirmations are UI-driven rather than cryptographically signed.
- Concurrency Handling: While the stored procedure uses SELECT FOR UPDATE for ticket locking, high-concurrency edge cases under real load would benefit from additional Oracle queue-based serialization.
- Session Management: The React frontend uses in-memory Context state. A browser refresh resets the session, requiring re-login. Persistent session storage (localStorage + server-side token validation) is absent.
- File Uploads: Event images, organizer avatars, and supporting documents cannot currently be attached to events or user profiles.
- Email Notifications: No email confirmation or reminder system is integrated for registration confirmations, payment receipts, or event reminders.

7.3 Future Work

Several enhancements are planned for future iterations of the system:

- Implement JWT-based stateless authentication with bcrypt password hashing and refresh token rotation.
- Integrate a real payment gateway (Razorpay or Stripe) with webhook-based payment confirmation callbacks.
- Add email notification triggers using Node.js Nodemailer for registration confirmations, waitlist promotions, and event reminders.
- Extend the data model to support recurring events, multi-day passes, and group registration.
- Build a mobile-responsive Progressive Web App (PWA) version using the same API backend.
- Implement advanced Oracle analytics: use window functions and materialized views for historical trend reporting in the Admin dashboard.
- Add QR-code-based digital ticket generation and scanner integration for physical event check-in.
- Deploy the full stack on a cloud platform (AWS / Azure) with Oracle Autonomous Database for production scalability.

CHAPTER 8: REFERENCES

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
2. Oracle Corporation. (2024). Oracle Database SQL Language Reference, 19c. Oracle Documentation. https://docs.oracle.com/en/database/oracle/oracle-database/
3. Oracle Corporation. (2024). Oracle Database PL/SQL Language Reference, 19c. Oracle Documentation. https://docs.oracle.com/en/database/oracle/oracle-database/
4. node-oracledb Development Team. (2024). node-oracledb: Oracle Database Driver for Node.js. GitHub Repository. https://github.com/oracle/node-oracledb
5. Meta Open Source. (2024). React Documentation. https://react.dev/
6. OpenJS Foundation. (2024). Node.js Documentation. https://nodejs.org/en/docs/
7. Express.js Team. (2024). Express - Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com/
8. Tailwind CSS. (2024). Tailwind CSS Documentation. https://tailwindcss.com/docs/
9. Vite. (2024). Vite Next Generation Frontend Tooling. https://vitejs.dev/
10. Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.